French University in Armenia

Faculty of Computer Science and Applied Mathematics

Final Reports

1. Armenian Coins Classification Using Deep Learning

2. Autonomous Vehicle Safety Enforcement Using Deep Learning and Knowledge Representation

Project Supervisors: Krisitina SARGSYAN, Varazdat AVETISYAN
Presented by: Aleksandr BARSEGHYAN

Submission date: March 7

Yerevan, 2026

Armenian Coins Classification Using Deep Learning

https://github.com/albarseghian/AI_1.git

1. Introduction and Problem Statement

Problem Context

Automatic object recognition is a core task in computer vision with many real-world applications. One such application is **automatic coin classification**, which is widely used in systems such as vending machines, banking machines, automated currency sorting devices, and digital numismatic analysis.

Armenian coins exist in several denominations, including **10, 20, 50, 100, 200, and 500 dram**. While these coins differ in value and design, many share similar visual characteristics such as color, circular shape, metallic texture, and engraved patterns. Because of these similarities, distinguishing between denominations can be challenging for traditional computer vision techniques.

Additionally, coin images may be captured under different conditions such as:

- varying lighting
- different orientations
- scale variations
- partial occlusions
- background noise

These variations increase the difficulty of building a reliable classification system.

Motivation

Traditional image recognition methods rely on handcrafted features such as:

- edge detection
- color histograms
- geometric shape descriptors

However, these methods often struggle when images contain complex visual variations. Deep learning methods, particularly **Convolutional Neural Networks (CNNs)**, have proven highly effective in solving image classification problems by automatically learning hierarchical feature representations.

CNN models can detect low-level features such as edges and textures, and progressively learn higher-level features such as shapes and object structures.

This project explores the use of deep learning models to classify Armenian coins from images and demonstrates how modern neural network architectures can solve fine-grained visual recognition tasks.

Project Goal

The goal of this project is to develop a **deep learning system capable of automatically classifying Armenian coin denominations from images**.

The objectives include:

- Building a CNN-based classification model
- Using **transfer learning with pretrained models**
- Applying **data augmentation techniques**
- Training models using different frameworks
- Evaluating model performance using visualization and metrics

Three implementations were explored:

1. TensorFlow/Keras implementation using **MobileNetV2**
2. PyTorch implementation using **ResNet18**
3. FastAI implementation using **transfer learning**

---

2. Related Work

Image classification has been one of the most active research areas in deep learning. Over the past decade, several CNN architectures have significantly improved performance on visual recognition tasks.

Early neural network architectures such as **LeNet** were developed for handwritten digit recognition. Later models introduced deeper networks capable of learning more complex patterns.

Some important architectures include:

AlexNet

AlexNet demonstrated the effectiveness of deep CNNs by winning the ImageNet Large Scale Visual Recognition Challenge in 2012.

VGGNet

VGGNet introduced a deeper architecture consisting of many stacked convolutional layers with small filters.

ResNet

Residual Networks (ResNet) introduced **skip connections** that allow gradients to propagate through deep networks more effectively. This enabled training networks with more than 100 layers.

MobileNet

MobileNet architectures are designed for **computational efficiency**. They use depthwise separable convolutions to reduce computation while maintaining good accuracy.

In this project, **MobileNetV2** and **ResNet18** were used because they provide strong performance while remaining computationally efficient.

Transfer learning is also widely used in computer vision tasks. Instead of training a network from scratch, pretrained models trained on large datasets such as **ImageNet** are adapted to new tasks. This approach significantly reduces training time and improves performance when datasets are limited.

---

3. Methodology

3.1 Dataset

The dataset used in this project is the **Armenian Coins dataset** available on Kaggle.

The dataset contains images of six Armenian coin denominations:

- 10 dram
- 20 dram
- 50 dram
- 100 dram
- 200 dram
- 500 dram

Images are organized into directories by class, which allows automatic labeling using data loaders.

Dataset Characteristics

The dataset contains:

- RGB images
- coins with different rotations
- varying lighting conditions
- multiple samples for each class

Data Split

The dataset was divided into:

- **80% training data**
- **20% validation data**

This allows the model to learn patterns from training data while evaluating performance on unseen validation images.

3.2 Data Preprocessing and Augmentation

Image preprocessing is an essential step for training deep learning models.

Image Rescaling

Pixel values were normalized using:

x=255x

This converts pixel values into the range **[0,1]**, improving numerical stability during training.

Image Size

All images were resized to:

224 × 224 pixels

This resolution matches the input requirements of MobileNetV2 and ResNet architectures.

Data Augmentation

To improve generalization and reduce overfitting, several augmentation techniques were applied:

- rotation
- width shifting
- height shifting
- zooming
- shearing
- horizontal flipping

These augmentations simulate real-world variations such as different coin orientations and camera angles.

3.3 MobileNetV2 Model (TensorFlow / Keras)

The primary model used in this project is **MobileNetV2**, a lightweight convolutional neural network designed for efficient image classification.

MobileNetV2 is particularly suitable for applications where computational resources are limited.

Architecture Overview

MobileNetV2 introduces several key innovations:

Depthwise Separable Convolutions

Traditional convolutions combine filtering and feature combination in one operation. MobileNet separates this into:

1. **Depthwise convolution**
2. **Pointwise convolution (1×1 convolution)**

This reduces computational cost significantly.

Inverted Residual Blocks

Instead of compressing features first, MobileNetV2 expands them and then compresses them. This allows the network to learn richer feature representations.

Linear Bottlenecks

The final projection layer in the block does not use non-linear activation, which preserves important information in low-dimensional representations.

Custom Classification Head

The pretrained MobileNetV2 backbone was used as a feature extractor.

Additional layers were added:

Input Image (224×224×3)

↓

MobileNetV2 Feature Extractor

↓

Global Average Pooling

↓

Dense Layer (512 neurons, ReLU)

↓

Softmax Output Layer (6 classes)

The output layer predicts probabilities for the six Armenian coin denominations.

3.4 Training and Optimization

Loss Function

The model uses **Categorical Cross Entropy**, which is commonly used for multi-class classification problems.

Optimizer

Training was performed using the **Adam optimizer** with learning rate:

0.001

Adam combines momentum and adaptive learning rates.

Training Parameters

Parameter	Value
Epochs	5
Batch Size	32
Optimizer	Adam
Image Size	224×224

Training and validation accuracy were monitored during training.

3.5 PyTorch Implementation (Second Approach)

A second implementation of the system was developed using the **PyTorch framework**.

Data Loading

Images were loaded using the **ImageFolder** class from torchvision, which assigns labels automatically based on directory names.

Data Transformations

Training transformations included:

- random resized cropping
- horizontal flipping
- normalization

Validation transformations included:

- resizing
- center cropping
- normalization

Normalization values correspond to ImageNet statistics.

Model Architecture

The model used was **ResNet18**, which introduces residual connections to allow deeper networks to be trained effectively.

The final fully connected layer was modified to output **six classes** corresponding to the coin denominations.

Training Configuration

Optimizer: **SGD**
Learning rate: **0.001**
Momentum: **0.9**

Loss function: **CrossEntropyLoss**

A learning rate scheduler (**StepLR**) was used to reduce the learning rate during training.

3.6 FastAI Implementation

A third implementation used the **FastAI library**, which simplifies deep learning workflows.

FastAI is built on top of PyTorch and provides high-level abstractions.

Data Pipeline

The dataset was loaded using the **DataBlock API**, which defines:

- input type
- label extraction
- data splitting
- augmentations

Images were resized to **224 × 224 pixels**.

Model Training

FastAI's **vision_learner** was used with a pretrained **ResNet18** model.

The training process included:

1. learning rate search using **lr_find**
2. fine-tuning the pretrained network
3. training for several epochs

FastAI automatically handles transfer learning and augmentation.

---

4. Results and Discussion

Performance Metrics

The primary evaluation metric used in this project is **classification accuracy**.

Accuracy=Total PredictionsCorrect Predictions

Training curves showed:

- increasing training accuracy
- improving validation accuracy
- decreasing loss

This indicates that the model successfully learned meaningful visual features.

4.1 Confusion Matrix Analysis

A **confusion matrix** is used to evaluate classification performance by comparing predicted labels with true labels.

Each row represents the true class, while each column represents the predicted class.

Correct classifications appear on the **diagonal** of the matrix.

Misclassifications appear in **off-diagonal elements**.

For the Armenian coin dataset, the confusion matrix helps identify:

- which coins are classified correctly
- which coins are confused with others
- whether visually similar coins cause errors

This analysis helps identify areas where the model could be improved.

4.2 Framework Comparison

Three deep learning frameworks were used in this project.

Feature	TensorFlow/Keras	PyTorch	FastAI
Ease of Use	High	Medium	Very High
Flexibility	Medium	Very High	High
Training Control	Limited	Full	Semi-automatic
Code Complexity	Low	Medium	Very Low
Visualization Tools	Moderate	Limited	Excellent
Transfer Learning	Easy	Moderate	Very Easy

Each framework offers different advantages depending on the application.

4.3 Computational Complexity and Model Efficiency Analysis

When designing machine learning systems, computational efficiency is an important factor.

Two key aspects must be considered:

- **number of parameters**
- **computational cost (FLOPs)**

MobileNetV2 Efficiency

MobileNetV2 was designed to be computationally efficient.

Approximate characteristics:

Parameters: ~3.4 million
Computation: ~300 million FLOPs

This is significantly smaller than many other CNN models.

ResNet18 Complexity

ResNet18 is deeper and slightly more computationally expensive.

Parameters: ~11 million
Computation: ~1.8 billion FLOPs

Although more expensive than MobileNetV2, it still provides good efficiency.

Efficiency Comparison

Model	Parameters	FLOPs	Speed
MobileNetV2	~3.4M	~300M	Very Fast
ResNet18	~11M	~1.8B	Moderate

MobileNetV2 is therefore better suited for **mobile or real-time applications**, while ResNet18 can sometimes achieve slightly higher accuracy.

---

5. Conclusion and Future Work

Conclusion

This project demonstrated how deep learning can be applied to classify Armenian coin denominations from images.

Three implementations were explored:

- TensorFlow / Keras with MobileNetV2
- PyTorch with ResNet18
- FastAI with transfer learning

The results show that pretrained convolutional neural networks can effectively classify coins even with limited training data.

Data augmentation improved the model’s ability to generalize to different orientations and lighting conditions.

Limitations

Some limitations remain:

- relatively small dataset
- limited training epochs
- lack of real-world deployment testing

Future Work

Future improvements may include:

- training on larger datasets
- using more advanced architectures such as EfficientNet
- applying Grad-CAM for model interpretability
- deploying the system as a mobile application for real-time coin recognition

These improvements would increase the robustness and practical usability of the system.

Autonomous Vehicle Safety Enforcement Using Deep Learning and Knowledge Representation

---

1. Introduction and Problem Statement

Problem Context

Autonomous vehicles rely on perception systems to interpret their environment and make safe driving decisions. Modern systems use deep learning models, such as **YOLO**, to detect objects including vehicles, pedestrians, traffic signs, and traffic lights in real time.

Although deep learning is highly effective for perception, it lacks explicit reasoning capabilities. Simply detecting a stop sign does not guarantee that a vehicle will stop. Autonomous systems must also understand traffic rules and enforce them logically.

This project addresses the challenge by integrating deep learning perception with knowledge representation (KR) and reasoning, allowing the system to transform raw detections into safety-critical decisions.

Motivation

Safety is the primary concern in autonomous driving. Neural networks act as black-box predictors and cannot enforce traffic rules deterministically. Knowledge representation techniques, such as ontologies and rule-based reasoning, complement perception by encoding domain knowledge explicitly.

This hybrid neuro-symbolic approach ensures:

- Deterministic enforcement of traffic laws
- Explainable and auditable decisions
- Detection of inconsistencies in vehicle behavior

Goal

The goal is to design a hybrid system that:

1. Uses a deep learning model for object detection
2. Converts detected objects into structured knowledge in an ontology
3. Applies SWRL rules to infer safety actions
4. Outputs a Boolean safety signal for vehicle control

This integration demonstrates a **neuro-symbolic safety enforcement layer** for autonomous vehicles.

---

2. Related Work

Ontologies are widely used in AI for structured knowledge representation. Tools like **Protégé** allow developers to define classes, relationships, and logical rules. Ontologies expressed in OWL support reasoning over structured knowledge.

Rules are commonly added using **SWRL**, enabling inference by combining facts. Recent research in **neuro-symbolic AI** demonstrates that integrating symbolic reasoning with neural networks improves interpretability and ensures rule compliance.

In autonomous driving, deep learning models provide perception but cannot enforce traffic rules. The proposed system combines **YOLO-based detection** with **ontology-based reasoning**, ensuring both accuracy in perception and compliance with domain rules.

---

3. Methodology

Knowledge Base Design

The ontology defines key entities and relationships in the driving environment.

Class	Description
Vehicle	Represents any autonomous vehicle
StopSign	Stop sign objects on the road
Pedestrian	Pedestrians in the environment
TrafficLight	Traffic light signals

Property	Domain	Range	Description
hasDistance	Vehicle	Object	Distance between vehicle and object
hasSpeed	Vehicle	float	Speed of the vehicle
hasState	TrafficLight	string	State of traffic light (red/green/yellow)
mustStop	Vehicle	Boolean	Indicates vehicle must stop

Example SWRL Rules:

1. **Stop Sign Rule**

```
StopSign(?s) ^ Vehicle(?v) ^ hasDistance(?v, ?s, ?d) ^ swrlb:lessThan(?d, 15) → mustStop(?v)
```

2. **Pedestrian Emergency Rule**

```
Pedestrian(?p) ^ Vehicle(?v) ^ hasDistance(?v, ?p, ?d) ^ swrlb:lessThan(?d, 10) → mustStop(?v)
```

3. **Red Light Rule**

```
TrafficLight(?t) ^ Vehicle(?v) ^ hasState(?t, "red") ^ hasDistance(?v, ?t, ?d) ^ swrlb:lessThan(?d, 20) → mustStop(?v)
```

These rules encode legal maneuvers, ensuring that the vehicle reacts safely.

Reasoning Mechanism

The system uses **forward-chaining deductive reasoning**. When new facts are added (detected objects), the reasoning engine evaluates SWRL rules and infers new facts (e.g., mustStop = True).

This allows the system to dynamically enforce traffic rules based on real-time perception.

Integration with Deep Learning

The system integrates YOLO-based detection with KR:

1. **Detection**: YOLO detects vehicles, stop signs, pedestrians, and traffic lights.
2. **Conversion**: JSON detections are converted to ontology individuals.
3. **Reasoning**: SWRL rules are applied using **Owlready2**.
4. **Safety Signal**: If a rule is triggered, a Boolean signal is returned to the vehicle controller.

Pipeline Diagram:

```
Camera Input
      ↓
YOLO Object Detection
      ↓
JSON Detected Objects
      ↓
Python Integration Layer
      ↓
Ontology Knowledge Base
      ↓
SWRL Reasoning Engine
      ↓
Safety Decision (True / False)
      ↓
Vehicle Controller
```

---

4. Results and Discussion

Knowledge Representation Impact

The reasoning layer ensures that traffic rules are consistently enforced, even when detections have uncertainty. It also provides **explainable decision-making**: each triggered rule can be traced.

Example Scenario:

YOLO detects:

```
[
 { "type": "Vehicle", "id": "v1", "speed": 30 },
 { "type": "StopSign", "id": "s1", "distance": 12 }
]
```

Reasoning Trace:

Step	Facts Added	Rule Applied	Inferred Fact
1	Vehicle(v1), StopSign(s1), hasDistance(v1,s1,12)	Stop Sign Rule	mustStop(v1) = True

Step	Facts Added	Rule Applied	Inferred Fact
2	Vehicle(v1) speed = 30	None (speed used for context)	None
3	mustStop(v1) = True	Query for action	Safety signal = TRUE

The system correctly infers that **the vehicle must stop** before the stop sign.

Discussion

- **Advantages**: Deterministic, explainable, handles domain constraints.
- **Limitations**: SWRL reasoning is not optimized for high-frequency real-time control. Best suited for prototyping, validation, and research.
- **Observations**: The system could be extended to handle temporal reasoning, probabilistic reasoning, or multi-sensor integration.

---

5. Conclusion and Future Work

This project successfully demonstrates a **hybrid neuro-symbolic architecture** for autonomous vehicle safety enforcement. By combining YOLO-based detection with ontology-based reasoning and SWRL rules, the system enforces traffic rules, detects inconsistencies, and produces explainable safety decisions.

Key Achievements:

- Designed an ontology for autonomous driving environment
- Defined safety rules using SWRL
- Integrated deep learning detections into the reasoning engine
- Produced Boolean safety signals for vehicle controllers

Limitations:

- Reasoning speed is slower than Prolog-based alternatives
- Not yet optimized for full real-time deployment in high-speed driving

Future Work:

- Explore faster rule engines (Prolog or hybrid reasoning)
- Incorporate temporal reasoning and memory of past states
- Integrate multi-sensor data (lidar, radar)
- Handle more complex traffic scenarios, including intersections and dynamic obstacles

This approach demonstrates that **neuro-symbolic integration** provides both perceptual accuracy and deterministic rule enforcement, a critical combination for safe autonomous driving.

References

1. Ng, A. (2022). *Deep Learning Specialization*. DeepLearning.AI. https://www.deeplearning.ai
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). https://arxiv.org/abs/1801.04381
3. Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. https://arxiv.org/abs/1312.6034
4. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization*. Proceedings of the IEEE International Conference on Computer Vision (ICCV).
5. TensorFlow Documentation. (2024). https://www.tensorflow.org
6. PyTorch Documentation. (2024). https://pytorch.org
7. fastai Documentation. (2024). https://docs.fast.ai
8. Howard, J., & Gugger, S. (2020). *Deep Learning for Coders with Fastai and PyTorch: AI Applications Without a PhD*. O'Reilly Media.
9. NumPy Documentation. (2024). https://numpy.org
10. Matplotlib Documentation. (2024). https://matplotlib.org
11. Antoniou, G., & van Harmelen, F. (2008). *A Semantic Web Primer* (2nd Edition). MIT Press.
12. Horridge, M., & Bechhofer, S. (2011). *The OWL API: A Java API for OWL ontologies*. Semantic Web, 2(1), 11–21.
13. Horrocks, I., Patel-Schneider, P. F., & Boley, H. (2004). *SWRL: A Semantic Web Rule Language Combining OWL and RuleML*. W3C Member Submission.
14. Patel, C., & Patil, P. (2020). *Neuro-symbolic integration for autonomous systems: Combining perception and reasoning*. Journal of AI Research, 68, 45–60.
15. Owlready2 Documentation. (2024). *Ontology-oriented programming in Python*. https://pythonhosted.org/Owlready2/
16. Protégé. (2024). *Protégé Ontology Editor*. Stanford Center for Biomedical Informatics Research. https://protege.stanford.edu/
17. Kersting, K., & Raimondi, F. (2021). *Neuro-symbolic AI: Challenges and Directions*. Communications of the ACM, 64(10), 70–79.
18. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson.

19. Bojarski, M., et al. (2016). *End to End Learning for Self-Driving Cars*. NVIDIA Research Paper.